

Contents

Reviewing a finished lesson	1
1. Mechanical	2
1.1 No figure sits directly under a heading	2
1.2 Every number in prose appears in a committed notebook output	2
1.3 Notation follows the table in CLAUDE.md	2
1.4 No em dashes in anything projected	2
1.5 The lesson plan's slide ranges match the segments	2
1.6 Every notebook is cited in the handout body, with its numbers	3
2. Judgement — the built PDF open, not the markdown	3
2.1 Open every figure and check the caption describes <i>that</i> image	3
2.2 Speaker notes point at the slide they are on	3
2.3 Slide titles are true, and carry the argument	3
2.4 Every term is defined before it is used	3
2.5 No claim promises more than the cell delivers	3
2.6 Criteria are stated at the moment they are applied	3
2.7 Look at the rendered deck	4
2.8 Every result is reached, not announced	4
2.9 The gap audit: read the deck as a student who has only the deck	5
3. The notebooks, which everything else quotes	6
3.1 The cell must compute the thing the sentence describes	6
3.2 A quoted number must be a property of the run, not of where it stopped	6
3.3 Dead code is evidence about what the author expected	6
3.4 Each method's own preconditions apply to it, including the lesson's	7
3.5 A number that motivates a technique must not be an artefact the lesson removes	7
3.6 Every API new to the lesson is named in prose, and linked	7
3.7 Re-run in the container, and check the figures did not move	7
3.8 Run every code block the prose prints, exactly as printed	7
Log	8
What check 2.8 found: every result reached, across all ten	15
What check 2.1 found in lesson 3	17
What check 2.1 found in lessons 4 to 10, and lesson 1	18
What check 2.7 found: the decks, looked at rather than counted	18
Fixed after that pass, still from reading the slides	19
What check 2.7 found in lesson 5, and in the second half of lesson 4	20
Still open across the course	20
4. When a number moves	24

Reviewing a finished lesson

`tools/verify_lesson.py` decides whether a lesson is *built*. This document is about whether it is *right*, which is a different question and is not automatable in full.

Every check below earned its place by catching something real in lesson 1 during the review of 28 August 2026. The examples are kept deliberately: a check without a failure it once caught tends to be skipped.

Work in this order. The mechanical checks are cheap and narrow the field; the judgement ones need the built PDF open in front of you.

1. Mechanical

1.1 No figure sits directly under a heading

Every figure must follow the paragraph that introduces it, so a reader meets the argument before the picture. A section that opens with a stack of images is a section whose figures were never placed.

Caught in lesson 1: thirteen of twenty-one figures. Section 7 opened with five pictures in a row, then the prose explaining them.

```
python tools/review_lesson.py NN          # reports every figure under a bare heading
```

1.2 Every number in prose appears in a committed notebook output

The handout and slides quote numbers the notebooks produce. Nothing but this check ties them together — `verify_lesson.py` compares `worked_examples.py` against the handout, and both can agree while the notebook beside them says something else.

Caught in lesson 1's neighbours: lesson 2's handout said AUC 0.752 where its notebook had long printed 0.751, and lesson 3's headline 247,514 had become 3,097,038,010 under the shipped image. Both lessons passed verification throughout.

1.3 Notation follows the table in CLAUDE.md

m is the number of examples, n the number of features, η the learning rate, λ the penalty. Slips are invisible one document at a time.

Caught in lesson 1: slide notes said the standard error “falls as one over root n ” where they meant m ; the quiz wrote the empirical risk as $(1/n) \Sigma$.

1.4 No em dashes in anything projected

Slide bodies, slide titles, metadata — and **inside figures**, where the text is drawn into the PNG and no search of the markdown will find it. Handouts keep theirs: an em dash in a typeset PDF is ordinary publishing.

Caught across the course: 839 in the decks, then a further 41 drawn inside figures, which the first pass could not reach.

1.5 The lesson plan's slide ranges match the segments

`verify_lesson.py` checks the cited slides *exist*, not that they are the right ones.

Caught in lesson 1: the plan cited 9–22, 23–30, 31–39, 40–51 for segments that actually began at 21, 31, 37 and 50.

1.6 Every notebook is cited in the handout body, with its numbers

Not only in the lesson plan table. A notebook nobody quotes is a notebook nobody runs.

Caught in lesson 1: notebook 02 appeared in the plan table, one homework line and three sets of speaker notes — and nowhere else. Handout section 5 had no numbers at all, the only section in the lesson arguing entirely in the abstract.

2. Judgement — the built PDF open, not the markdown

2.1 Open every figure and check the caption describes *that* image

The most productive check in the whole list, and the one no tool can do.

Caught in lesson 1: three captions described figures that had never been made. `error_costs.png` was promised as “two problems side by side” and shows one. `overfitting.png` was captioned as two error curves and shows three polynomial fits. A slide’s notes said “follow the two curves” of a histogram.

2.2 Speaker notes point at the slide they are on

A note describing a figure that is on the *next* slide is the standard failure, because notes get appended while the author is thinking about what comes after.

Caught in lesson 1: five of them.

```
python tools/review_lesson.py NN          # flags deictic notes with no figure beside them
```

2.3 Slide titles are true, and carry the argument

Caught in lesson 1: “The same idea, on real data” over a figure fitted to 22 synthetic points. Retitled to what its own notes tell the lecturer to point at.

2.4 Every term is defined before it is used

Students cannot look it up mid-lecture. Expand acronyms, and define words that look ordinary but are not.

Caught in lesson 1: R^2 used four times and never defined — nor anywhere else in the ten lessons. `n_init=10` with no explanation. “cultivar” used seven times.

2.5 No claim promises more than the cell delivers

Caught in lesson 1: the self-supervised section said the learned representation “transfers to tasks you do care about” and demonstrated no transfer at all. The shortcut-feature section said the column “is empty” at deployment, implying an error that in practice never comes.

2.6 Criteria are stated at the moment they are applied

A test the reader can only run after the fact is not a criterion.

Caught in lesson 1: “errors are catastrophic and unexplainable” as a reason *not to build* a model described a model already deployed and already failing.

2.7 Look at the rendered deck

```
soffice --headless --convert-to pdf deck.pptx
pdftoppm -png -r 55 deck.pdf slide
```

Overlapping annotations, a legend across a curve, a cropped axis label, a title wrapping to two lines: none of these fail a check, and all are obvious in the render.

2.8 Every result is reached, not announced

The handout is the one artefact that carries derivations, so a result it states without a route to it has nowhere else to be explained. This check is about *consequentiality*: whether a reader who accepts each sentence is carried to the next one, or has to supply a step the text skipped.

It is not a licence to add material. Nothing new is taught — the same results, with the bridge between them written down.

Four failures, in the order they are worth hunting for:

A result quoted instead of derived. The worst kind, because it is invisible: the sentence reads as authoritative and the student cannot tell that a step is missing, only that they are lost a page later.

Caught in lesson 6: the margin “is $2/\|w\|$ ” — no distance formula, no canonical normalisation, so the ≥ 1 in the constraints arrived from nowhere. And “its solution depends on the data only through inner products”, the single fact the entire kernel trick turns on, given by decree.

One algebraic step skipped. Usually an expansion or a cancellation the author did in their head.

Caught in lesson 5: §5.2 jumps from $\mathbb{E}[(f + \varepsilon - \hat{f})^2]$ straight to $\sigma^2 + \mathbb{E}[(f - \hat{f})^2]$, leaving the reader to expand the square and argue the cross term away.

A choice made without saying why that one. A definition, a loss, a normalisation introduced as if it were the only candidate.

A silent monotone step. Replacing a quantity by its logarithm, its square, its reciprocal, without the half-sentence saying the ordering is preserved.

Caught in lesson 6: §4.2 moves from the arg max of a product to the arg max of its logarithm with no mention that the logarithm is increasing.

What the repair costs, and what it must not cost. New algebra is new intermediate values, which is exactly how lesson 3’s 165,200 happened. Every step added here goes into `Docs/worked_examples.py`, checked against a route that does not reuse it — against a fitted model, a simulation, or a second derivation. Lesson 6’s rework added 17 such checks: that the solver really does place the closest point at $|w^\top x + b| = 1$, that $2/\|w\|$ equals the gap measured geometrically, that `dual_coef_` rebuilds `coef_`, that the RBF kernel equals its truncated power series.

Three things move with the text, and all three are gated: the lesson’s notation table, the concepts index (new vocabulary earns an entry), and the Italian translation in `Review/`, which

`tools/check_dispenza.py` will fail the moment a number appears in one language and not the other. The front matter’s reading time is *not* gated and has to be updated by hand.

2.9 The gap audit: read the deck as a student who has only the deck

Run it the week before the lesson is taught, every lesson. Every other check here was written from reading the material. This one was written from teaching it: lesson 1 went into a room on 2 October 2026, and in the following three days the questions students asked, and the moments they stopped following, exposed gaps that two full reviews had passed. Auditing lesson 2 against the same list before it was taught found eight of them.

Read the slide **bodies** only, in order, notes closed, and ask of each slide whether a student who missed the spoken part could follow it. The handout usually has the answer; the slides usually do not, and the slides are what is on the wall when the question is asked.

Eight kinds of gap, each with what it looked like:

1. **A term used before it is defined.** Most often a metric, because it feels too familiar to define. > Lesson 1 said “accuracy” from its first slide and defined it nowhere. Lesson 2 > quoted every result of its second half as an AUC, and only the handout said what > an AUC is.
2. **An acronym expanded only in the notes.** `verify_lesson.py` now checks the slide body alone, which is the mechanical half of this; a figure that shows bare letters is the half it cannot see. > Lesson 2’s MCAR, MAR and MNAR were spelled out in the notes and nowhere the room > could see them, including the figure that introduced them.
3. **A named method without its formula.** A student shown the name and the result cannot check the result. > Lesson 1 named precision and recall without $TP / (TP + FN)$. Lesson 2 compared > StandardScaler with MinMaxScaler without $(x -) /$ or $(x - \min) / (\max - \min)$.
4. **A process described but never stated, or never shown.** The formal statement, then the smallest concrete instance of it. > Lesson 1 never said what “learning” is: it needed $f = \arg \min R(f)$ on one slide > and a threshold searched across 355 candidates on the next.
5. **A step whose origin is not said.** Something appears, and nobody says where it came from. > Lesson 1’s confusion matrix arrived as labels, and the probability behind each > label, compared with an unchosen 0.5, came only in the notes.
6. **One symbol or one number for two things, or two names for one.** > Lesson 1 used p for both the true and the measured accuracy. Lesson 2 used k for > the z-score cut-off and for a count of categories; said “100:1” on one slide and > “110:1” on another for the same ratio, built and measured; and called the same > quantity “step size” and “learning rate”.
7. **A claim without its condition.** > Lesson 1’s “training error falls monotonically with flexibility” holds for nested > families and a solver that finds the best member; `np.polyfit` at degree 21 does > not. “Touched once” was read as “because the measurement is precise”. > Lesson 2’s slides said a missingness indicator “keeps MAR signal alive” while > its own handout measured $0.751 \rightarrow 0.741$ with one and concluded the opposite; > the audit missed it, and the instructor’s own study session caught it. When the > handout qualifies a rule, check that the slide stating the rule carries the > qualification.
8. **Code the notebook runs that no slide names.** `verify_lesson.py` checks this one (`check_code_anchors`): every scikit-learn name a notebook calls must appear on a slide, in the toolbox table of its “Notebook N, live” slide and as an “In code:” line where the concept is introduced. Add the lesson to `ANCHORED` when its anchors are written. > Lesson 1’s notebooks

called 22 scikit-learn names and its slides showed 4; `> train_test_split` was met for the first time inside a code cell.

How to repair, and what not to break. - Prefer folding a gap into an existing slide over adding one. No new slide means no renumbering of the plan table, the toolbox slide references or the Review documents. - A display equation stays alone on its slide; its symbols are named in the notes. - The build’s “runs past the bottom” failure is the usual price of a fix; shorten a neighbouring bullet rather than drop the fix. - A fix to the handout’s wording goes into `Review/dispensa_it.md` too. - Keep the misconception slides: a slide that states the predictable wrong answer on purpose, corrected by the next one, is not a gap. Lesson 2’s “MCAR: safe to impute” is one.

3. The notebooks, which everything else quotes

Sections 1 and 2 check the material *around* the code. This one checks the code, and it is the pass that matters most, because the handout, the slides and the quiz all quote numbers these cells produced. A wrong cell is wrong in five places at once, and `verify_lesson.py` will pass throughout — `worked_examples.py` recomputes what the *handout* prints, so a handout faithfully reprinting a bad notebook number agrees with itself perfectly.

Run every notebook in the container, then work the list below.

3.1 The cell must compute the thing the sentence describes

Not something close to it. A proxy that lands near the right answer is the hardest version of this to catch, because nothing looks wrong.

Caught in lesson 2: the lesson’s carry-home number, “98 of 128 imputed rows borrowed from the test set”, came from a neighbour search over 3 standardised columns. The `KNNImputer` that actually imputed used 4 raw columns, the `nan_euclidean` metric, and only rows that had an `age` to donate. Measuring what the sentence claimed gave 94.

3.2 A quoted number must be a property of the run, not of where it stopped

Before reprinting a value, ask what it would be after twice as many iterations.

Caught in lesson 2: “the cost reaches 3.34, far above where it started. It diverges.” The loss does not diverge — it oscillates between 0.69 and 8.29 and stays there, and 3.34 was simply where the bounce happened to be at step 200. The mean over the first hundred steps is 3.64 and over the second hundred 3.04; at 2000 steps it is still in the same band.

3.3 Dead code is evidence about what the author expected

A branch that cannot fire usually marks a belief the data did not support.

Caught in lesson 2: an `if len(losses) < n_iter` branch printing “diverged after N steps” — unreachable, because the run never breaks early. It was written by someone who expected a divergence that does not happen, and it sat directly above the prose claiming one.

3.4 Each method’s own preconditions apply to it, including the lesson’s

The lesson teaches a rule in section 5 and then breaks it in section 9 with no comment.

Caught in lesson 2: `KNNImputer` fitted on unscaled columns, where the standard deviations run 184, 54, 13 and 1.1 — so “nearest neighbour” meant “similar bill” and `num_support_calls` contributed nothing. Section 5 of the same lesson is the argument that distance-based methods need scaling first.

3.5 A number that motivates a technique must not be an artefact the lesson removes

Caught in lesson 2: scaling was motivated by `monthly_charges` having “3-4 times” the spread of `tenure_months`. That ratio is 3.6 only because of the 20 billing errors section 4 exists to find; cleaned, it is **0.8**, the other way round. The slide notes said 0.8 and the notebook said 3-4, and neither knew about the other.

3.6 Every API new to the lesson is named in prose, and linked

Students cannot look up a call they have only seen inside a code cell, and the conceptual explanation being good is not a substitute — it tells them what is happening, not what to type into a search box.

Caught in lessons 1 and 2: zero documentation links across all six notebooks. `train_test_split`, `make_pipeline`, `predict_proba`, `cross_val_score`, `f_classif`, `get_dummies` and `roc_auc_score` appeared only in code, several of them on first encounter in the whole course.

3.7 Re-run in the container, and check the figures did not move

Byte-identical figures confirm the committed images came from the same environment. Anything that moves without an edit to the plotting code is a reproducibility problem worth understanding before it reaches a student.

3.8 Run every code block the prose prints, exactly as printed

A block that carries its own `import` lines is a promise that it is complete. Paste it into a fresh interpreter and see. The same goes for the loading snippet an exercise hands out: if the prose describes the data, the snippet has to produce that data.

Half of this is now automatic. `tools/check_code_blocks.py`, run by `verify_lesson.py` inside the container, parses every printed block and binds each call against the real signature: too many positional arguments, a keyword that does not exist, a missing required one, a name a module does not export, a block that does not parse. It cannot run the fragments — a slide’s four lines use variables built three slides earlier — so **what it does not check is still yours**: that the names a block leaves undefined are ones the reader has, and that the snippet produces the data the prose then describes. Both of the findings below are of that second kind.

Caught in lesson 5, by the checker that did not exist then: the deck printed `SelectKBest(f_classif, 10)`, a `TypeError` since `k` became keyword-only, in the one artefact students copy from — while the handout and notebook beside it both wrote `k=10`. It survived a full review pass, because reading a call does not tell you its signature.

Caught in lesson 2: the handout’s `ColumnTransformer` block used `numeric_columns` and `categorical_columns`, defined nowhere, and called `LogisticRegression` without importing it — three `NameErrors` in a block printing its own imports.

Caught in lesson 1: the exercise’s `fetch_openml` call returns columns named `V1...V11` and quality scores renumbered **1–7**, while the brief promises eleven named physico-chemical measurements and “an integer from 3 to 9” and asks students to justify a threshold on that scale. Nothing errors; the assignment is simply written against data the snippet does not load.

Log

What has actually been through this pass, so nobody redoes it or assumes it was done.

Lesson	Date	Result
1	28 Aug 2026	Arithmetic clean. Every headline number recomputed independently and reproduced exactly: baseline 0.629 and model 0.986 on breast cancer; wine 0.981, R^2 0.622 and 0.816, adjusted Rand index 0.897; the <code>n_init=1</code> study (6 distinct solutions, 0.322–0.915); the 30-seed split spread 0.917–1.000 and 5-fold 0.960 ± 0.030 . Fixed: no documentation links anywhere and seven APIs never named outside code cells; one imprecise comparison (“a little over one point”, “five times”) replaced with the measured 1.4–2.3 and 6.8–8.1. Code blocks (3.8): the handout has none; the exercise’s loader contradicted its own brief, and the setup slide’s <code>git clone</code> wrapped twice — the second wrap inside the URL with no backslash, so anyone copying it got a broken command.

Lesson	Date	Result
2	28 Aug 2026	Three real errors, all listed above as 3.1–3.5: the 98-of-128 proxy, the false divergence claim with its dead branch, and <code>KNNImputer</code> on unscaled columns. Carry-home number moved to 94 of 128 and propagated to handout, slides, quiz, figure and <code>CLAUDE.md</code>. Notebook 1’s arithmetic was clean. Code blocks (3.8): three undefined names in the handout’s pipeline block.
2 (2nd pass)	28 Aug 2026	Worked the exercise as a student, on seed 104. Tasks 1–5 and 6(b) are sound. Task 6(a) was a trap: over 15 seeds the imputation leak’s effect on AUC has mean +0.0001 and comes out negative in 7 of 15, while Task 7 asked students to explain why the numbers differ, with 30% of marks on that explanation. Neither lever the material offered opens the gap — <code>age</code> does not appear in the generator’s churn model at all, so contaminating it cannot move a score. Rewrote 6(a), Task 7, the hints and the rubric around that finding; corrected the same false promise in handout §9.1 and in a “Try this” this reviewer had introduced earlier the same day. Cross-lesson references: all resolve, one overstatement fixed. Quiz: numbers current, §7 had no question at all — added. Resources: three GDPR list omissions. Speaker-note timings realigned to the plan.

Lesson	Date	Result
3	28 Aug 2026	Complete. Section 3.2 claimed notebook 1 “does the same thing on 450 houses and gets 2,410” — a number in no notebook. The same one-feature fit gives 2,785, further from the truth than the three-house hand example, because area absorbs bedrooms (+0.77) and bathrooms (+0.49); all six features give 2,421. Notebook 2 still described a “degree-15 fit on 84 training points” where it fits degree 12 on 21 — one of the four errors CLAUDE.md already records as historic. Section 4.4 said “design matrix” for a table computed without the intercept column, where its own Section 3.1 defines the design matrix as including one (654, not 285). The deferred conditioning paragraph is written, with the ridge conditioning table recomputed from singular values — the eigenvalues of $X'X$ are numerical noise at 4.2e9, which would have been an unfortunate way to get a number wrong in a section about conditioning. Fifteen figures moved under the prose that introduces them. <code>worked_examples.py</code> from 17 checks to 37. Quiz: question 17 was built on pre-restack numbers (247,514 / 182 / 16.3, none of which exist anywhere any more) and the quiz used for both the learning rate and the penalty, four questions apart — the exact collision CLAUDE.md’s notation table names lesson 3 for. Slides: the same 2,410 claim, and “118 at degree 9, 182 at degree 12” where the table now reads 590 and 24,656. Exercise: the brief called an hourly dataset “daily

Lesson	Date	Result
4	28 Aug 2026	Complete, and the cleanest so far. Every worked number in the handout recomputes: the cost arithmetic ($140/2740 = 0.051$, $13 \times 140 + 33 \times 2600 = 87,620$), the odds multipliers as exp of the fitted coefficients, $(-6.09) = 0.0023$, and the AUC identity checked against 200,000 sampled pairs. The 33-question quiz is accurate throughout, including its harmonic-mean arithmetic. Two structural fixes: thirteen figures placed under the prose that introduces them, and <code>precision_recall_curve.png</code> moved out of §6.1 — it plots precision against recall with thresholds marked, and sat two sections before any of those three words was defined. Section 6 opened straight onto a subsection and now has an introduction. Exercise worked end to end: the decoy is identifiable (-0.005 against a true 0), theoretical and empirical thresholds agree to 0.039 vs 0.045, and the imbalance test delivers its point cleanly — AUC $0.931 \rightarrow 0.922$ while precision falls $0.256 \rightarrow 0.078$. Resources checked; references sound, nothing changed.

Lesson	Date	Result
5	28 Aug 2026	Complete. No wrong numbers in any artefact — every figure the handout quotes traces to a notebook, and the 29-question quiz is accurate. The gap was verification: <code>worked_examples.py</code> had six checks, the fewest in the course, for the lesson that is <i>about</i> verification. Now nineteen, with section 2.2's whole table (0.885 / 1.000 / 0.955 / 0.024, ten of two hundred above 0.99) rebuilt from the generator rather than transcribed. Eight figures placed under their introducing prose. The exercise was worked end to end and is the best-designed in the course so far: three planted flaws, of which the two everyone predicts are worth 0.006 between them and the third — 12 readings per turbine split at random — is worth 0.037 and multiplies the fold spread by six. Resources checked; references sound. One finding for the whole repository: <code>StratifiedGroupKFold</code> assigns groups differently between scikit-learn 1.3.1 and 1.9.0, so this exercise's honest figure is 0.881 in the image and 0.868 outside it. Verification done in the host venv is not verification.

Lesson	Date	Result
6	28 Aug 2026	Complete. The /C table in section 5.4 — six numbers the handout and the slides both print — was computed only inside three figure titles and printed nowhere, so nothing compared it against anything. All six are correct (verified against the notebook’s own fold seed; a different seed gives 0.930 and 0.893, which is how the gap showed up). Notebook 3 now prints the table, and <code>worked_examples.py</code> recomputes it: 20 checks to 26. Section 4.5’s “reporting 0.999” read as a measurement of this dataset sitting beside two real ones; it needs correlated features, which the pump data has none of, so it now says so and gives the conditions under which it happens (ten near-copies of one column: accuracy 0.75, two thirds of predictions above 0.999). No figures out of place.
7	28 Aug 2026	Complete, no wrong numbers. The depth sweep, bagging, boosting and the 1/e bootstrap limit all recompute against the notebooks. The final leaderboard’s second column — each score as a fraction of the 0.93 noise ceiling — is arithmetic the handout does in prose and nothing checked; now gated. 38 checks to 46.

Lesson	Date	Result
8	28 Aug 2026	Complete, no wrong numbers. 1,500 sessions with 180 bots leaves the 1,320 humans the text quotes, DBSCAN’s 13 noise points really are all human, and the scree eigenvalues sum to the 8 standardised columns. Section 5.4 opened on the scree plot before saying what a scree plot is for; it now has an introduction and the figure follows the table.
9	28 Aug 2026	Complete, and careful. No wrong numbers, and the handout does something none of the others do: it quotes the <i>median</i> end-to-end gradient ratio (3,547) across eight seeds, states the 2,734–4,607 range, and warns in the text against quoting the largest as though it were a law. Six figures placed under their prose; section 11.2 opened on two figures back to back and now says what question they answer. One speaker note said “the figure is in handout section 7.4” with a different figure on the next slide — disambiguated.
10	28 Aug 2026	Complete, no wrong numbers. Every parameter count checks: $576 \times 256 + 256 = 147,712$ dense against $8 \times 9 + 8 = 80$ convolutional, so the slides’ “147,632 missing parameters” and the ratio of 1,846 are both exact. Section 3.3 opened on a figure; it now says why one writes kernels down before learning them.

What check 2.8 found: every result reached, across all ten

One pass, 13 September 2026, prompted by a reader reporting that lessons 5 and 6 were hard to follow in places and that lessons 1 and 2 had the same feel. The finding is that the difficulty was **not** thinness: it was that the course is uneven about the *middle* of an argument. Lesson 9’s backpropagation shows every intermediate step, checks the shapes and points out the cancellation; lesson 6’s support vector machine, three lessons earlier, announced two results and derived neither.

What was repaired, lesson by lesson.

Lesson	Section	What was announced, and is now reached
1	2.3	The unbiasedness of a test score, now in one line of linearity of expectation — and the sentence saying which equality fails when the test set is consulted early.
1	2.4	$\sqrt{p(1-p)/m}$: one yes/no draw has variance $p(1-p)$, and averaging m of them divides it by m .
2	3.2	$\text{Var}(X') = (1-p)\sigma^2$: the section promised “only the bookkeeping” and then omitted it.
2	5.2	Removed “stated rather than derived”. It was worse than that — lesson 3 claimed <i>lesson 2</i> had derived the stability threshold, which lesson 2 never states. Now an honest forward pointer.
2	9.2	The target-encoding identity: the weighted-blend line the subtraction acts on.
3	4.3	$\alpha < 2/c$, derived from the contraction factor $(1 - \alpha c)$, with all three learning-rate regimes read off one expression.
3	4.4	Why iterations grow with the condition number: spend $1/c_{\max}$ and the shallow direction contracts by $1 - 1/\kappa$, so $t \approx \kappa$.

Lesson	Section	What was announced, and is now reached
4	4.3	“We state without full proof” is gone: the Hessian from section 3.4’s gradient, then positive semi-definiteness in one line.
4	6.3	What a harmonic mean <i>is</i> , and where β^2 comes from.
4	7.2	The rearrangement into t^* .
5	2.4	The rarer-class rule, from the standard error of a proportion over m_+ draws — with the AUC case argued separately rather than assumed.
5	4.3	$\text{Var}(\hat{R}_{CV})$, from the variance of a sum of correlated terms.
5	5.2	The three-term expansion, and why the cross term dies.
5	4.7	Why cross-validation is seven times more stable. A first attempt at this was wrong — it decomposed the factor into $\sqrt{k}$ and $\sqrt{29/7}$, whose product is 4.5, and then explained away a gap in the direction that arithmetic cannot produce. Replaced with the mechanism that is actually true: what the seed can move differs between the two procedures.
6	3.2	Distance concentration: the centre grows like n , the spread like $\sqrt{n}$.
6	4.2	The logarithm is increasing, so the arg max does not move.
6	5.2	Where $2/\ w\ $ comes from — distance to a hyperplane, then the canonical normalisation that puts the 1 in the constraints.

Lesson	Section	What was announced, and is now reached
6	5.4	The dual: multipliers, $w = \sum_i a_i y_i x_i$, and the data appearing only inside inner products. Without it the kernel trick was a decree. Plus why the RBF is infinite-dimensional.
7	2.2	Why $1 - \sum p_c^2$ is the chance two draws disagree.
7	7	The binomial cancellation to k/p .
7	5.2	Cross-referenced to lesson 5's §4.3 — the same arithmetic twice in the course, now said so.
8	6.1	Why $V_k V_k^\top x$ is the <i>closest</i> point of the subspace: orthogonality, then Pythagoras — lesson 3's normal equation again.
9	8.2	$\text{Var}(z) = n \text{Var}(w) \mathbb{E}[a^2]$, in its two steps, and the ReLU's half.
10	3.2	The output-size formula counted rather than quoted: the last legal start is $n + 2p - f$.
10	4.1	Equivariance in one line of index arithmetic.

What it cost. The handouts grew 1–4%, except lesson 5 (+10%) and lesson 6 (+30%, from the thinnest in the course to about the median). Reading times updated by hand for those two and for lesson 3. No slide changed: derivations belong in the handout, which is the whole reason it exists.

What gates it. 70 new checks in `worked_examples.py`, taking the course from 585 to 655 — every one reaching the new step by a route that does not reuse it — the SVM margin measured geometrically against $2/\|w\|$, `dual_coef_` rebuilding `coef_`, the RBF kernel against its truncated power series, the projection against a thousand rival points of its own subspace, Hessians by finite differences, standard errors by simulation. Lesson 2 is the one that gained none, and correctly: its repairs wrote out bookkeeping for identities its checks already covered, introducing no new number. Concepts indices gained entries for the new vocabulary, and all ten Italian translations in `Review/` were updated in the same pass, so `check_dispenza.py` still passes on every one.

What check 2.1 found in lesson 3

Recorded because the faults have a shape, and it is not the one the check was written for. Only one was a caption describing the wrong picture; three were figures that did not show what they claimed,

and the worst was a stale number nobody could see.

- **alpha_trade_off.png** had its six points hard-coded, and one was pre-restack. The leftmost said test error 182.0 where the truth is 17.3 — and 182.0 is one of the two values removed from the quiz the same morning. That single number manufactured the entire left arm of the U and put the marked optimum in the wrong place: the real minimum is $= 10$, not 0.01. The figure now computes its own sweep over ten orders of magnitude.
- **learning_rate_regimes.png** contradicted the rule its own section derives. Section 4.3 states convergence requires $< 2/c$, the bowl has $c = 1$, and the panel titled “Too large: it climbs out” used $= 1.96$. It converged. Now 2.15.
- **train_test_by_degree.png** showed none of what its caption promised. On a linear axis the degree-12 value of 24,656 flattened every other point onto the x-axis, so neither the falling training error nor the turn at degree 3 was visible. Log scale.
- **coefficient_trust.png** carried a caption for a different figure — “the same coefficient across four random splits”, where the image is six features’ error against their correlation with `area_sqm`. Two of its labels also overlapped.
- **regularisation_paths.png** and **alpha_trade_off.png** labelled the penalty `.CLAUDE.md` reserves for the learning rate. The quiz and exercise were corrected earlier the same day; the figures draw it into the PNG, where no search finds it. **regularisation_paths.png** also titled a panel “everything shrinks” over a curve that triples first.

What check 2.1 found in lessons 4 to 10, and lesson 1

Worth recording because the result is mostly negative, and a negative result from this check is the expensive kind to obtain.

- **No caption anywhere in lessons 4, 6, 7, 8 or 10 misdescribes its figure.** Every number drawn into those images traces to a notebook: lesson 4’s three ROC operating points and its multiclass matrix whose rows sum to the classification report’s supports; lesson 6’s three support vectors, all three of which sit at $|\text{decision}| = 1.000$ exactly; lesson 7’s box plots and bootstrap limit; lesson 8’s elbow and silhouette both landing on $k = 4$; lesson 10’s permutation bars.
- **Lesson 5 needed two.** Its high-variance learning curve was captioned “still climbing” where it rises to 250 examples and then flattens, and section 6.1 built its advice on that slope. Its bias-variance figure was captioned “bias falls with complexity” over a curve that climbs four orders of magnitude — which the prose three paragraphs below already explained, so the caption was asserting what the text had to walk back.
- **Four legend or label placements**, in lessons 3, 4, 9 and 10, each sitting over the curve or bar the figure exists to compare.
- **One cross-lesson attribution**, found while checking item 6: lesson 10 called its dense baseline “lesson 9’s dense network on the same images: two layers of 256”. Lesson 9’s digit networks are one and two hidden layers of 64 units on 8×8 images. The 213,761 parameters are right for lesson 10’s own 24×24 images; the attribution was not.

What check 2.7 found: the decks, looked at rather than counted

Added 29 August 2026, after rendering all ten decks to one PNG per slide and reading them. Three faults, none of which any existing check could see.

- **Twenty slides ran off the bottom of the slide**, all in lessons 7 to 10 and all the same shape: five bullets that each wrap to two lines. The last line either sat behind the university crest or was cut off by the edge. The check in `postprocess_pptx.py` was supposed to catch this and did not - it counts characters, and scored all thirteen of the worst slides *identically* before and after they were fixed. `verify_lesson.py` now rasterises the built PDF and looks for ink below the footer line, which is the thing a student sees. On the previous commit it names eight slides in lesson 9 alone.
- **Multi-word `\text{}` lost its spaces in every equation image**. Lesson 2 slide 32 projected `onlyiffisdependentofT`. `render_math` maps `\text` to `mathtext's \mathrm`, which sets the argument as one run; it now sets one `\mathrm` per word. Lesson 8's "chosen next" and lesson 10's "output size" were the other two. Inline maths had the mirror problem, a space the control word only needed as a terminator: `$\Delta G$` came out as " ΔG ".
- **One slide carried bullets and a display equation together** - lesson 6's soft-margin formulation, the layout `CLAUDE.md` warns about. The build claimed to warn about it and only checked for text *after* an equation; it now checks for a bullet list beside one. Exactly one slide in the course tripped it.

Also found here, though it belongs to item 6: **three lessons still quoted 98 of 128 imputed rows**, which this review corrected to 94 in lesson 2. Lessons 8, 9 and 10 each repeat lesson 2's number in a carry-home list, and nothing checked that a number quoted from another lesson still matched its source.

`verify_lesson.py` now does, in both directions: a lesson that no longer contains the number `CLAUDE.md` remembers it by, and a lesson that reproduces another's carry-home wording beside a number that appears nowhere in the lesson it came from. Run against the commit that corrected lesson 2, it names all three stale quotations; run against the course as it stands, it says nothing.

Fixed after that pass, still from reading the slides

The three faults above were what the first pass found. Reading further into the decks turned up six more, each invisible in the markdown and obvious in the render.

- **`\hat` was mapped to the empty string, so $\hat{y}$ projected as y** . Lesson 9's shapes table is the one place inline maths uses the prediction, and exactly the place the distinction from the target matters. `render_math` now uses Unicode's precomposed circumflex where one exists and the combining one for the rest.
- **Half-converted subscripts printed as products**. Lesson 7 projected $F_t(x)$ and ht , lesson 8 r_j : the conversion lowered the characters it had a glyph for and left the rest at full size. The guard only fired when *nothing* converted, which was worse still — `m_L` came out as `mL`. The script tables now carry every raised and lowered letter Unicode has, and a formula that still cannot be fully converted is refused whole, so the build names it rather than shipping something misleading. That left eight formulas with no honest inline form: lesson 7's were in speaker notes and are now words, and lesson 9's shapes table says "W, layer 1" with the real notation in the display equations above it.
- **Lesson 2's slide 6 carried three bullets and a four-panel figure**, which left the histograms a strip a fifth of the slide high — while their own speaker notes open "do not describe shapes the room cannot see". Split, with the plot alone on "Two of these four have no shape left to read"; the lesson plan's slide numbers move on by one from there.
- **Lesson 3's by-hand table wrote X X open in four places** where the equation two slides earlier sets it closed up, and its $X y$ row carried a stray space before the comma.

- **Two legends in lesson 10 covered the reading their slide titles quote.** `sample_efficiency.png` sat its legend over the dense curve's last segment, hiding the value at 8,000 images under the title "0.235 short at 8,000"; `family_comparison.png` covered the logistic-regression pair on "Four hand-written numbers reach the ceiling", one of the four. Moved into the empty band between the curves, and below the axes in two columns.
- **Lesson 4's slide 17 asked a question without a question mark** — "So why not squared error", the title of the slide whose whole job is to answer it.

What check 2.7 found in lesson 5, and in the second half of lesson 4

Read 29 August 2026, closing the two decks the first sweep had rendered and not read. Lesson 5 is the lesson about verification, so it is the one where an unread deck was least comfortable; it had five faults, and none of them was a wrong number.

- **Two annotations written on top of the bars they describe.** `mean` 0.955 sat at 55% of the axis height, which put black text across two bars of the histogram, while its own siblings `worst` 0.885 and `best` 1.000 sit above the bars. And `what pure chance would give` was centred on $x = 1.5$, so half of it lay on the $C = 1$ bar — the one bar the dashed chance line exists to be compared against. Both moved into white space.
- **`n = 25 | n = 60 | n = 150` for the number of examples**, in the handout's section 5.4 table, on the slide that repeats it, and in quiz question 16. `CLAUDE.md` reserves `n` for features and `m` for examples, and this lesson contradicted itself twice over: its own notation table gives `m` as the number of examples, and slide 16 of the same deck says "`k = m` is leave-one-out". This is lesson 3's collision in a lesson that had not been suspected of it.
- **A table whose columns did not add up.** Slide 24 printed `bias2`, `variance` and `total`, having dropped the `noise` column the handout carries — so the arithmetic on screen was short by 484 in every row, and only the speaker notes said why. The column is back; five columns fit a slide comfortably.
- **A legend across the confidence band it belongs to**, in the left panel of the learning curves, where everything above 0.80 is empty. `loc="lower right"` was being applied to both panels by one loop; it is right for the panel on the left only.

The second half of lesson 4 — 32 slides — is clean: no legend over a curve, no overlapping annotation, no cropped label, and every number that appears on a slide checks. The threshold sweep's five rows are internally consistent ($TP + FN = 76$ in each), both confusion matrices total 2,000, the three-class matrix's rows give 1,665 / 259 / 76, and F1's harmonic 0.073 against arithmetic 0.519 is exact. The only fault was the missing question mark recorded above.

Two things deliberately left: lesson 5's slide 32 wraps a code block inside a call, and its slide 36 prints 0.7999 beside 0.500 in one column. Both are legible, and neither is worth a rebuild on its own.

Still open across the course

Written down 28 August 2026, after the notebook pass finished. Ordered by what each is worth.

#	What	Where it stands
1	Check 2.1 — open every figure, confirm the caption describes <i>that</i> image. The most productive check in this document, and the one no tool can do.	Done for all ten lessons. The faults cluster hard: five each in lessons 2 and 3, none of substance anywhere else. Lessons 4–8 and 10 needed only four legend or label placements; lesson 5 needed two captions that stated the textbook version of what the figure complicates; lesson 1’s three faults were found and fixed in the August review and the fixes have held.
2	Check 2.7 — look at the rendered deck.	Done for all ten, now genuinely. Every deck rendered to one PNG per slide and read to the end: eight on 29 August, then lesson 5 and the second half of lesson 4, which the first sweep had rendered without reading. Twenty overrunning slides, three equation-rendering faults, one bullets-beside-an-equation slide, six more from reading further, and five in lesson 5 — all fixed, and the sections above record them. Three now have an automated check that did not exist before, so those cannot return silently.
3	CLAUDE.md’s “one number per lesson” table stops at lesson 3.	Done. All ten rows filled, each with a figure verified during this review rather than chosen from memory.

#	What	Where it stands
4	Two submission conventions. Exercises 1–3 were due “at the start of Lesson N+1”; exercises 4–10 at “23:59” on the same Friday, and neither form named an hour.	Closed, 29 August 2026, by removing the deadline rather than choosing one. The instructor’s model is: the exercise is set at the end of lesson N, discussed at the start of lesson N+1, and nothing is collected or marked weekly — at the exam one of the ten is drawn and the student talks through their own notebook. So there is no submission to be late for, and the exercises are assessed once, out loud. Rewritten through Assessment/ (all three READMEs), the ten exercises, the ten handouts, the ten decks and their speaker notes, SCHEDULE.md, SYLLABUS.md and CLAUDE.md. Two things fell out of it: the exercises carried two file-naming conventions ({surname}_01.ipynb against exercise04_<surname>.ipynb, the same 1–3 / 4–10 split as the dates), which disappears when nothing is handed in — they are all exerciseNN.ipynb now; and Assessment/Exercises/README.md promised “solutions are published after the due date in this folder”, which contradicted CLAUDE.md’s rule that solutions are never committed. The discussion is the solution.

#	What	Where it stands
5	Deck density. Originally written as “46 to 66 slides per lesson against roughly 80 lecture minutes: 35–50 slides an hour, where CLAUDE.md says 25–30”.	Measured 29 August 2026, and the original claim was wrong. Those 80 minutes were an estimate; each lesson’s own plan budgets 99 to 160 minutes of lecture once notebooks and the break are removed. Against the plans, the course runs at 26.5 slides per lecture-hour, inside the band. Only lessons 8, 9 and 10 exceed it, at 31.7, 33.9 and 32.8, and four lessons sit under 25 — lesson 5 at 19.2 and lesson 4 at 20.2. What is worth deciding is not the lesson average but two segments: lesson 9’s “Initialisation, optimisers, regularisation” at 12 slides in 13 minutes (55/h) and lesson 4’s “Cost-optimal thresholds; multiclass; homework” at 9 in 10 (54/h). A minute a slide. Closed 12 September 2026: left as it is, by the instructor’s decision, and the measurement behind it is worth keeping. Four of lesson 9’s twelve slides are figures — one word of body each — so counting slides overstates that segment: by words it is about seventeen minutes of material in a thirteen-minute box, long by four rather than by half. Nothing in it is spare (initialisation, optimisers and regularisation are all used by notebook 3, which follows immediately), merging a claim onto its figure is what build.py’s overflow check exists to reject, and no redistribution of minutes helps because the lesson runs at 36.8 slides per lecture-hour however the minutes are spread. The remedy, if one is ever wanted, is in the room and not in the deck

#	What	Where it stands
6	Cross-lesson claims. Each lesson makes checkable assertions about the others — 144 references in all.	Done for the falsifiable subset, and the carry-home numbers are now gated. Every reference naming a section number or quoting a figure was checked: four of them, three correct and one wrong (lesson 10’s attribution of its dense baseline to lesson 9). The remaining 140 are prose pointers of the form “lesson 5 returns to this”, which resolve but carry nothing to falsify. The numbers that <i>are</i> repeated verbatim — the carry-home list — are checked against their source lesson by <code>verify_lesson.py</code> on every run.
7	Four decks hand back an exercise on the day it is due. Lessons 7 to 10 opened with “Exercise N returned”, and lesson 7’s exercise 6 was due that same morning — no interval in which to mark it, under either deadline. Lessons 2 to 6 did the opposite and collected the exercise due that day.	Closed, 29 August 2026, by the same decision as item 4. Nothing is collected and nothing is handed back, so all nine openings are now discussions: “Exercise N, before we start” or “which we discuss now”, with speaker notes that ask the room rather than telling the lecturer to collect or return. The bullets kept their content — what mattered in that exercise and the gap to listen for — since it was always about the exercise due that day, and only the framing was wrong.

4. When a number moves

If re-running a notebook changes a number the material quotes, decide whether the number is *stable* before reprinting it. A figure that depends on an ill-conditioned solve will move again at the next image rebuild.

Lesson 3: the largest coefficient of the degree-12 fit is deterministic inside the image

and moved by a factor of 12,500 when the linear-algebra backend changed — the design matrix is full rank with condition number $4.2\text{e}9$. The lesson's carry-home number was moved to one that had not budged.